Video Transcript

Video 6: Using PySpark to Query Flight Data (07:29)

Now that we have a Spark system up and running that we have some data, let's go ahead and do some analysis of that data using SQL, one of the languages that is supported by Spark. We'll start off by creating a Spark session and I'll give you a walkthrough through the steps that we're going to be doing, and then later we'll come back and write the code in detail. The very first thing that we'll do is that we'll read the data and we'll infer some of the schema, we'll create a table as well, we'll look for the flights that are greater than 1000 miles as an example. We'll then look at some of the delays. This is one example. Here is another one where we're trying to see how long of a delay we have. So, as mentioned, this is just a quick overview.

Let's go back and walk through that code one by one, and we'll start off by creating a session. So, as you can see here, I am at the console, on the Spark shell, the Python Spark shell. I'm going to go ahead and clear my screen here, and I'll get started here by creating a session as you can see there. Well, at this point, I'm simply importing the package. Next, I'm going to go ahead and create that session, I'm going to name it Departuredelays. Next, I'm going to go ahead and create a variable that will hold the path to our data, the departuredelays.csv file, the file that has over a million rows. Next, we're going to go ahead and read the data and as you can see here, there's a few points worth noting. One that the format we're telling it to expect CSV, another option there is that we are inferring a schema. That is, take a look at the file, make your best guess, and as you can see there, that will be good enough for what we're trying to do, this is a pretty simple file structure.

And, so let's go ahead and execute that, and as you can see there, that takes a moment. We have quite a bit of data in that file, but you can see there that, that now has gone through. Now, we're going to then, based on that, we're going to go ahead and read that into a view. As you can see here, you can think of it as a table. So, now that we have a session, that we've loaded the data and that we have a table, we can go ahead and start to write some queries. And as you can see there, that is the SQL syntax that you would expect. We're selecting and we're specifying there the columns that we're looking for, the distance, origin, destination from delays

© MIT xPRO® 2024. All rights reserved.

table. And if we go back, you can see here where we specified that table, that view, we gave it a name. And then, where the distance is greater than 1000, we're then going to order by, we're going to do an aggregation and you can see there by distance and descending order. We're simply going to show, we're only going to show 10 there.

So, let's go ahead and hit 'return'. And as you can see there, we get a result nicely formatted onto the console. So, it's worth noting here what took place. We uploaded a CSV file. We, then, infer the schema from it. That is, we do not have to go through the usual exercise of being able to, of defining tables, and creating data types and so on. And then, we were able to write a query in SQL against that data. So, you can see that there, that's pretty speedy, pretty fast to be able to do this. Now next, let's say we were interested in flights that had delays of over two hours between a couple of well known airports, that is the San Francisco airport and the Chicago airport. So, let's go ahead and enter another query. And as you can see there, we're selecting for date, delay, origin, and destination from the delays table where delays are greater than 120 and the origin is San Francisco and the destination is Chicago. We're going to order once again and let's go ahead and return. So, as you can see there, we get the delays, we get the origin and destination, and we get the dates as well. So, once again, pretty handy functionality, great combination there of SQL, Python, and the platform of Spark.

Now, one more in this case, we want to get a sense of the types of delays that we have. And, I've gone ahead and pasted the query onto the command there as onto the console. You can see there that we're selecting, once again, the delay, the origin, and the destination. But in this case, we're looking at a case statement and we are going to report on very long delays, long delays, short delays, tolerable delays, and so on to be able to get a sense of what this data looks like. Now initially, I'll run it quickly just this is there and you can see there the data that is coming back. But, let's go ahead and update that so that we can see more of the data. Now as we do that, you can see here that we start to get tolerable. We get some earliest, some short and some long. So, you can start to get there more of a sense of the data. It's easier to parse, certainly visually, and you can see, there the type of statement that we wrote.

Now, to illustrate the flexibility, we have been working on the traditional SQL model, but to illustrate that we can do the same with a more traditional use of a data frame. Let's go ahead and import that package and then write the very first query that we did, but do it using the dataframe. And as you can see there, we can select and we can specify there the columns and

© MIT xPRO® 2024. All rights reserved.

then you can do the filtering. As you can see there, the WHERE clause and then the ORDER BY. And if we hit 'return', you can see there that we get the same output that we originally got when we did the SQL query, which I'll repeat here again just so we can have it next to each other. Well, I wasn't quite able to get it next to each other, but I think you see there the comparison of both. So, now it's your turn. You can flex your SQL muscles, go ahead and dig into this data. This is a fun dataset. You can post many more questions and you can explore here within the shell what you can do.

© MIT xPRO® 2024. All rights reserved.